

Assignment 1

(20 points)

Written Assignments

1. Do the following changes to a computer system increase throughput, decrease response time, or both? (3 points)
 - a. Replacing the processor in a computer with a faster version
 - b. Adding additional processors to a system that uses multiple processors for separate tasks—for example, searching the web.
 - c. Discuss other hardware and software factors that can affect computer performance.
2. Some great ideas in computer architecture are similar to ideas from other fields. Match the following ideas from computer architecture, “Use Abstraction to Simplify Design”, “Make the Common Case Fast”, “Performance via Parallelism”, “Performance via Pipelining”, “Performance via Prediction”, and “Dependability via Redundancy” to the following ideas from other fields. (4 points)

You must explain what these ideas are and match them to one of the following applications in other fields.

- a. Assembly lines in automobile manufacturing
 - b. Suspension bridge cables
 - c. Aircraft and marine navigation systems that incorporate wind information
 - d. Express elevators in buildings
 - e. Library reserve desk
 - f. Increasing the gate area on a CMOS transistor to decrease its switching time
3. Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2. (4 points)
 - a. Which processor has the highest performance expressed in instructions per second?
 - b. If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
 - c. We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?
 4. Assume a program requires the execution of 50×10^6 FP instructions, 110×10^6 INT instructions, 80×10^6 L/S instructions, and 16×10^6 branch instructions. The CPI for each type of instruction is 1, 1, 4, and 2, respectively. Assume that the processor has a 2 GHz clock rate. (5 points)

- a. By how much must we improve the CPI of FP instructions if we want the program to run two times faster?
 - b. By how much must we improve the CPI of L/S instructions if we want the program to run two times faster?
 - c. By how much is the execution time of the program improved if the CPI of INT and FP instructions is reduced by 40% and the CPI of L/S and Branch is reduced by 30%?
5. When a program is adapted to run on multiple processors in a multiprocessor system, the execution time on each processor consists of computing time and the overhead time required for locked critical sections and/or to send data from one processor to another. (4 points)

Assume a program requires $t = 100$ s of execution time on one processor. When run on p processors, each processor requires t/p s, as well as an additional 4 s of overhead, irrespective of the number of processors. Compute the per-processor execution time for 2, 4, 8, 16, 32, 64, and 128 processors. For each case, list the corresponding speedup relative to a single processor and the ratio of actual speedup to ideal speedup (speedup if there was no overhead).

Processor	T = 100 for 1 Processor	T with overhead	Speedup (compared to one CPU)	Speedup ratio (compared to no overhead)
2				
4				
8				
16				
32				
64				
128				

Notes on CPU Performance and Its Factors

Formula for the execution time for a program related to the most basic metrics (clock cycles and clock cycle time) and CPU time:

CPU execution time for program = CPU clock cycles for program x Clock cycle time

Alternatively, because clock rate and clock cycle time are inverses,

CPU execution time for program = CPU clock cycles for program / Clock rate

We can express the basic performance equation in terms of instruction count (the number of instructions executed by the program), CPI (Cycles per Instruction), and clock cycle time:

CPU execution time for program = Instruction count of program x CPI x Clock cycle time

Since the clock rate is the inverse of clock cycle time:

$$\text{CPU execution time} = \text{Instruction count} \times \text{CPI} / \text{Clock rate}$$

One alternative to time is MIPS (million instructions per second). For a given program, MIPS is simply

$$\text{MIPS} = \text{Instruction count} / (\text{Execution time} \times 10^6)$$

Since MIPS is an instruction execution rate, MIPS specifies performance inversely to execution time; faster computers have a higher MIPS rating. The good news about MIPS is that it is easy to understand, and faster computers mean bigger MIPS, which matches intuition.

There are three problems with using MIPS as a measure for comparing computers. First, MIPS specifies the instruction execution rate but does not take into account the capabilities of the instructions. We cannot compare computers with different instruction sets using MIPS, since the instruction counts will certainly differ. Second, MIPS varies between programs on the same computer; thus, a computer cannot have a single MIPS rating. For example, by substituting for execution time, we see the relationship between MIPS, clock rate, and CPI:

$$\text{MIPS} = \text{Clock rate} / (\text{CPI} \times 10^6)$$

There is a reliable method of determining and reporting performance by using the execution time of real programs as the metric. This execution time is related to other important measurements we can make by the following equation:

$$\text{Seconds / Program} = (\text{Instructions / Program}) \times (\text{Clock cycles / Instruction}) \times (\text{Seconds / Clock cycle})$$

Installing a C Compiler and Code Editor

In this course, we will use a C compiler to compile and run C programs. Mac users will use Clang, while Windows users will use GCC through WSL/Ubuntu.

We will use Visual Studio Code (VS Code) as the recommended code editor. VS Code is free and available for macOS and Windows.

Mac / macOS

Installing a C Compiler

On macOS, we will use **Clang**, the C compiler included with Apple's Command Line Tools. You do **not** need to install Homebrew or GCC.

1. Open **Terminal**.
2. Enter the following command:

```
xcode-select --install
```

3. A window will appear asking you to install the Command Line Tools. Follow the instructions to complete the installation.
4. After the installation is complete, verify that the compiler is installed by entering:

```
clang --version
```

WINDOWS OS

1. Installation instructions can be found here.
<https://docs.microsoft.com/en-us/windows/wsl/install>

If you are using an older version of Windows, then you have to manually install WSL, WSL2 and then install Ubuntu.

<https://docs.microsoft.com/en-us/windows/wsl/install-manual>

2. After installing Ubuntu, you need to install the GCC compiler.

```
sudo apt update  
sudo apt install build-essential  
gcc --version
```

3. Once you have installed Ubuntu on Windows, you can review some common best practices for using Ubuntu with WSL.
<https://docs.microsoft.com/en-us/windows/wsl/setup/environment>